M3 – PRACTICA 1:

Informe Técnico: Auditoria y Estandarización del inventario multisede

FECHA: 18 DE OCTUBRE DE 2026

Enlace al Git: https://github.com/Gazirr/Master-BigData-e-IA.git

GABRIEL HERRADOR

Índice

1.- Introducción y Objetivos 2
1.1.- Contexto del Proyecto: 2
1.2.- Objetivos 2
2.- Metodología de Auditoria y Limpieza 3
3.- Análisis y Auditoria Columna por Columna (Fase de Auditoria) 4
Columna 1- Product_id: 4
Columna 2 – Nombre: 5
Columna 3 – Categoría: 7
Columna 4 – Proveedor: 8
Columna 5 – Precio Unitario: 10
Columna 6 – Stock Actual: 12
Columna 7 – Stock Mínimo: 13
Columna 8 – Almacen: 15
Columna 9 – Fecha Ultima Entrada: 16
Columna 10 – Estado: 18
4.- Fase de limpieza Automática Multisede 20
4.1.- Propósito y Estrategia de Limpieza: 20
4.2.- Reglas de Transformación por Columna: 20
5.- Unificación e Integración del dataset Maestro 22
5.1.- Concatenación y Trazabilidad Multisede: 22
5.2.- Resultado del Dataset Consolidado: 22

1.- Introducción y Objetivos

1.1.- Contexto del Proyecto:

El presente proyecto aborda la necesidad de auditar, limpiar y unificar los datos de inventario provenientes de cuatro sedes heterogéneas: **Barcelona (BCN)**, **Madrid (MAD)**, **Sevilla (SEV)** y **Valencia (VAL)**. Cada sede gestiona su propio archivo de inventario con inconsistencias estructurales, registros duplicados, valores nulos y falta de estandarización en los formatos de texto y numéricos.

1.2.- Objetivos

- **Auditoria Exahustiva:** Evaluar la cantidad de los datos columna por columna, tomando el dataset de Barcelona (BCN) como modelo de referencia

- **Diseño de Pipline de limpieza unificado:** Crear una lógica de limpieza masiva y automatizada aplicable a los cuatro ficheros de forma idéntica, garantizando la escalabilidad y reusabilidad del código.

- **Exportación y Normalización:** Generar un conjunto de datos consolidado, limpio y sin anomalías, listo para su explotación analítica y toma de decisiones.

2.- Metodología de Auditoria y Limpieza

Se ha definido una arquitectura de trabajo basada en cuatro fases **(Load-Analyze-Clean-Export):**

1. **Carga Estructurada:** Lectura centralizada de los ficheros mediante una función de carga automatizada (**cargar_inventario**) almacenando cada dataset en un diccionario global.
2. **Auditoría de Calidad:** Análisis detallado de cada columna bajo la plantilla estandarizada **Propósito - Problemas encontrados - Solución**.
3. **Limpieza Masiva:** Aplicación de funciones de transformación y validación sobre la totalidad del diccionario de sedes.
4. **Consolidación y Exportación:** Verificación de integridad y guardado de los datos normalizados.

3.- Análisis y Auditoria Columna por Columna (Fase de Auditoria)

Columna 1- Product_id:

Propósito: Disponer de un identificador único que distinga de forma univoca cada uno de los registros(filas) contenidos en los diferentes archivos de inventario

Problemas encontrados: Aunque en el archivo de muestra (inventario_BCN.csv) la columna se encuentra completamente íntegra y sin duplicados, al auditar de forma global el resto de sedes se detectan anomalías puntuales (valores nulos y registros duplicados en algunos ficheros como Madrid o Sevilla).

Solución: Se establecerá una regla general en la función de limpieza automatizada para que, independientemente de que un dataset esté limpio o presente incidencias, se verifiquen y corrijan los valores nulos y duplicados del identificador en los cuatro ficheros del diccionario de manera simultánea.

```
----Analisis de la Columna: Producto_id ----

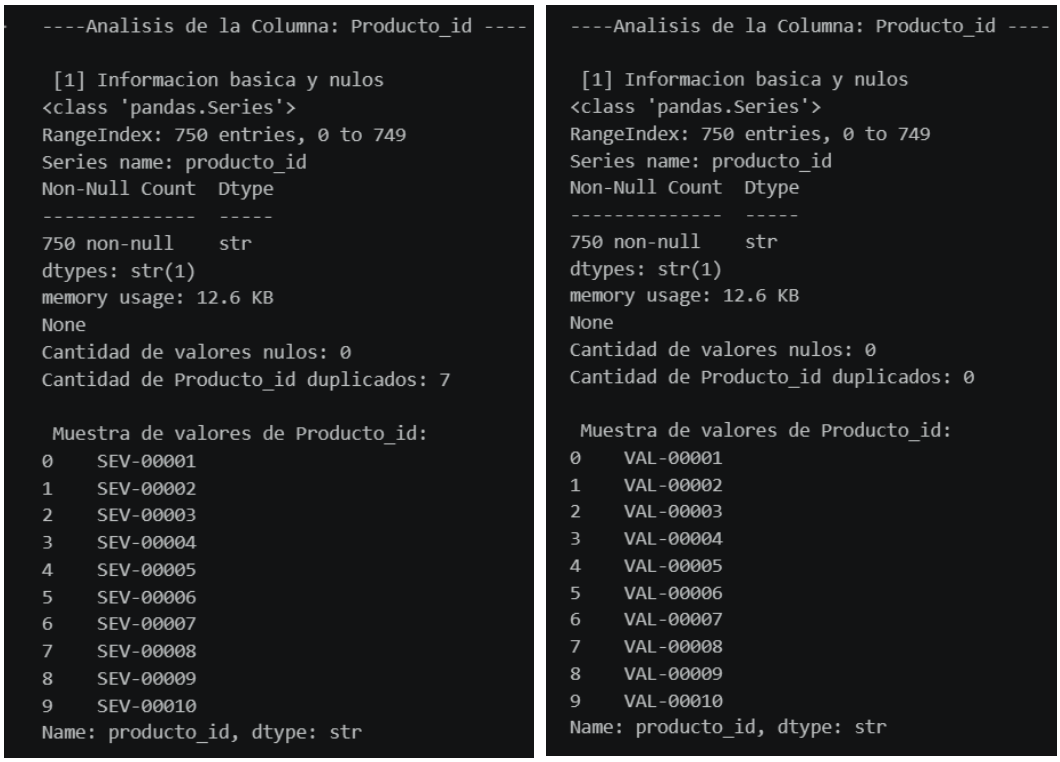
 [1] Informacion basica y nulos
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: producto_id
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 12.6 KB
None
Cantidad de valores nulos: 0
Cantidad de Producto_id duplicados: 0

 Muestra de valores de Producto_id:
0    BCN-00001
1    BCN-00002
2    BCN-00003
3    BCN-00004
4    BCN-00005
5    BCN-00006
6    BCN-00007
7    BCN-00008
8    BCN-00009
9    BCN-00010
Name: producto_id, dtype: str
```

```
----Analisis de la Columna: Producto_id ----

 [1] Informacion basica y nulos
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: producto_id
Non-Null Count  Dtype
--------------  -----
747 non-null    str
dtypes: str(1)
memory usage: 12.6 KB
None
Cantidad de valores nulos: 3
Cantidad de Producto_id duplicados: 7

 Muestra de valores de Producto_id:
0    MAD-00001
1    MAD-00002
2    MAD-00003
3    MAD-00004
4    MAD-00005
5    MAD-00006
6    MAD-00007
7    MAD-00008
8    MAD-00009
9    MAD-00010
Name: producto_id, dtype: str
```



```
----Analisis de la Columna: Producto_id ----

 [1] Informacion basica y nulos
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: producto_id
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 12.6 KB
None
Cantidad de valores nulos: 0
Cantidad de Producto_id duplicados: 7

 Muestra de valores de Producto_id:
0    SEV-00001
1    SEV-00002
2    SEV-00003
3    SEV-00004
4    SEV-00005
5    SEV-00006
6    SEV-00007
7    SEV-00008
8    SEV-00009
9    SEV-00010
Name: producto_id, dtype: str
```

```
----Analisis de la Columna: Producto_id ----

 [1] Informacion basica y nulos
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: producto_id
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 12.6 KB
None
Cantidad de valores nulos: 0
Cantidad de Producto_id duplicados: 0

 Muestra de valores de Producto_id:
0    VAL-00001
1    VAL-00002
2    VAL-00003
3    VAL-00004
4    VAL-00005
5    VAL-00006
6    VAL-00007
7    VAL-00008
8    VAL-00009
9    VAL-00010
Name: producto_id, dtype: str
```

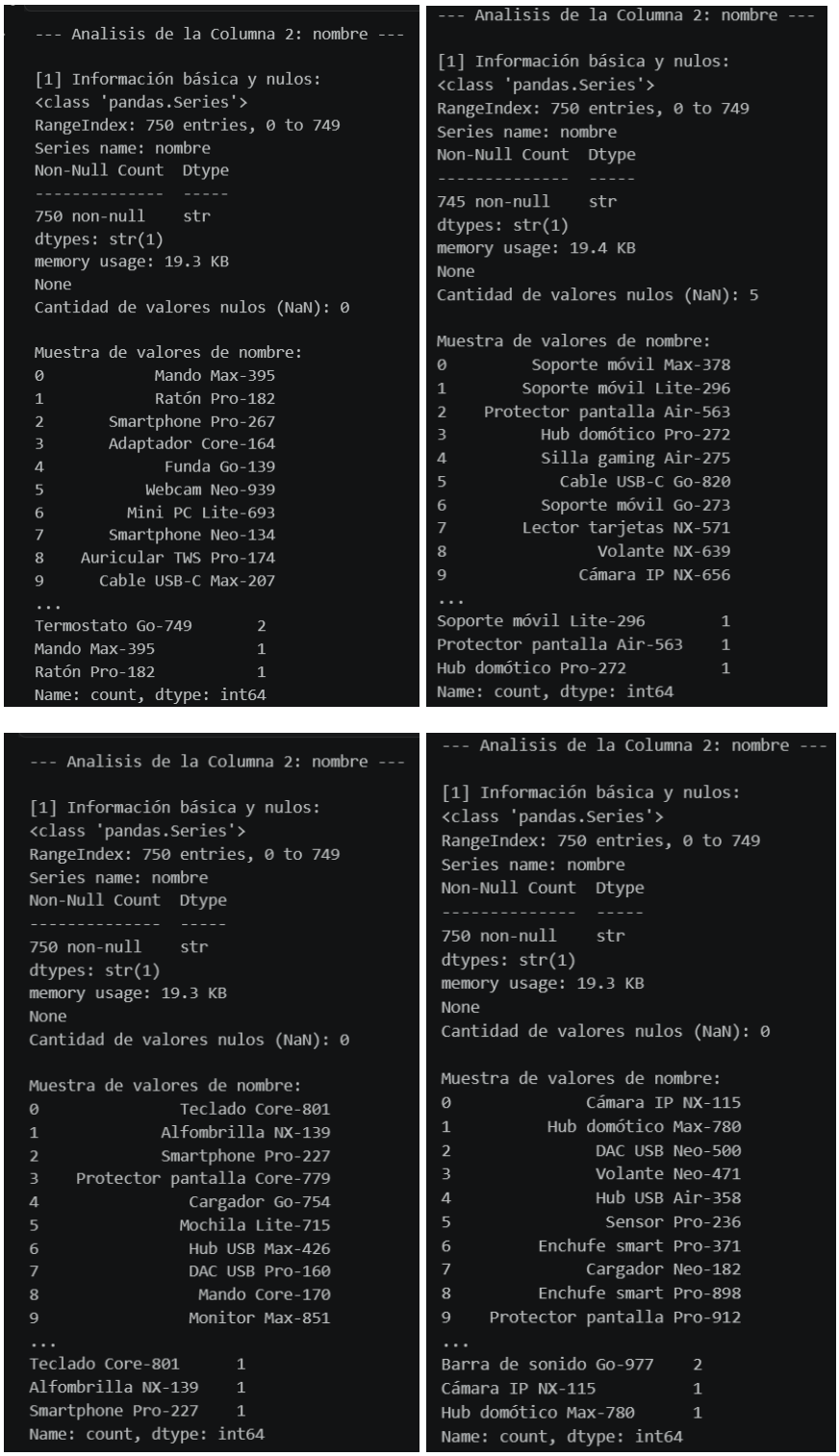
Figura 1: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 2 – Nombre:

Propósito: Almacenar la denominación o descripción comercial del producto para su correcta identificación dentro del catálogo unificado.

Problemas encontrados: El análisis multisede revela diferencias significativas en la integridad de este campo. Mientras que en Barcelona, Sevilla y Valencia la columna está completa (750 registros sin nulos), en Madrid se detectan 5 valores nulos (NaN). Además, al tratarse de un campo de texto libre ingresado manualmente en diferentes almacenes, existe riesgo de espacios en blanco sobrantes (trim/strip) y ligeras divergencias tipográficas.

Solución: Se programará en la función de limpieza masiva el tratamiento automatizado para imputar o gestionar los nulos en Madrid, junto con la eliminación de espacios iniciales y finales (.str.strip()) en todas las sedes.



```
--- Analisis de la Columna 2: nombre ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: nombre
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 19.3 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de nombre:
0            Mando Max-395
1            Ratón Pro-182
2       Smartphone Pro-267
3       Adaptador Core-164
4             Funda Go-139
5           Webcam Neo-939
6         Mini PC Lite-693
7       Smartphone Neo-134
8    Auricular TWS Pro-174
9      Cable USB-C Max-207
...
Termostato Go-749       2
Mando Max-395           1
Ratón Pro-182           1
Name: count, dtype: int64
```

```
--- Analisis de la Columna 2: nombre ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: nombre
Non-Null Count  Dtype
--------------  -----
745 non-null    str
dtypes: str(1)
memory usage: 19.4 KB
None
Cantidad de valores nulos (NaN): 5

Muestra de valores de nombre:
0         Soporte móvil Max-378
1        Soporte móvil Lite-296
2    Protector pantalla Air-563
3          Hub domótico Pro-272
4          Silla gaming Air-275
5            Cable USB-C Go-820
6          Soporte móvil Go-273
7        Lector tarjetas NX-571
8                Volante NX-639
9              Cámara IP NX-656
...
Soporte móvil Lite-296        1
Protector pantalla Air-563    1
Hub domótico Pro-272          1
Name: count, dtype: int64
```

```
--- Analisis de la Columna 2: nombre ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: nombre
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 19.3 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de nombre:
0               Teclado Core-801
1             Alfombrilla NX-139
2             Smartphone Pro-227
3    Protector pantalla Core-779
4                Cargador Go-754
5               Mochila Lite-715
6                Hub USB Max-426
7                DAC USB Pro-160
8                 Mando Core-170
9                Monitor Max-851
...
Teclado Core-801      1
Alfombrilla NX-139    1
Smartphone Pro-227    1
Name: count, dtype: int64
```

```
--- Analisis de la Columna 2: nombre ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: nombre
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 19.3 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de nombre:
0              Cámara IP NX-115
1          Hub domótico Max-780
2               DAC USB Neo-500
3               Volante Neo-471
4               Hub USB Air-358
5                Sensor Pro-236
6         Enchufe smart Pro-371
7              Cargador Neo-182
8         Enchufe smart Pro-898
9    Protector pantalla Pro-912
...
Barra de sonido Go-977    2
Cámara IP NX-115          1
Hub domótico Max-780      1
Name: count, dtype: int64
```

Figura 2: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 3 – Categoría:

Propósito: Clasificar los productos en agrupaciones comerciales estandarizadas (Hogar inteligente, Informática, Gaming, Telefonía, Accesorios, Audio) para análisis sectoriales y filtrado de inventario.

Problemas detectados: Se observan problemas tanto de integridad como de formato. En Sevilla se identifica 1 valor nulo (NaN) e inconsistencias de capitalización como INFORMÁTICA en mayúsculas. Por su parte, Madrid, Barcelona y Valencia presentan los 750 registros completos, pero requieren asegurar la homogeneidad categórica.

Solución: La función de limpieza convertirá el campo a formato título/minúsculas normalizado, eliminará espacios redundantes e imputará el valor faltante en la sede de Sevilla de forma transparente.

```
--- Analisis de la Columna 3: categoria ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: categoria
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 13.4 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de categoria:
0         Gaming
1    Informática
2      Telefonía
3     Accesorios
4      Telefonía
5    Informática
6    Informática
7      Telefonía
8      Telefonía
9      Telefonía
...
Hogar inteligente    126
Telefonía            123
Audio                114
Name: count, dtype: int64
```

```
--- Analisis de la Columna 3: categoria ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: categoria
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 13.5 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de categoria:
0            Telefonía
1            Telefonía
2           Accesorios
3    Hogar inteligente
4               Gaming
5            Telefonía
6            Telefonía
7           Accesorios
8               Gaming
9    Hogar inteligente
...
Hogar inteligente    126
Informática          125
Gaming               123
Name: count, dtype: int64
```

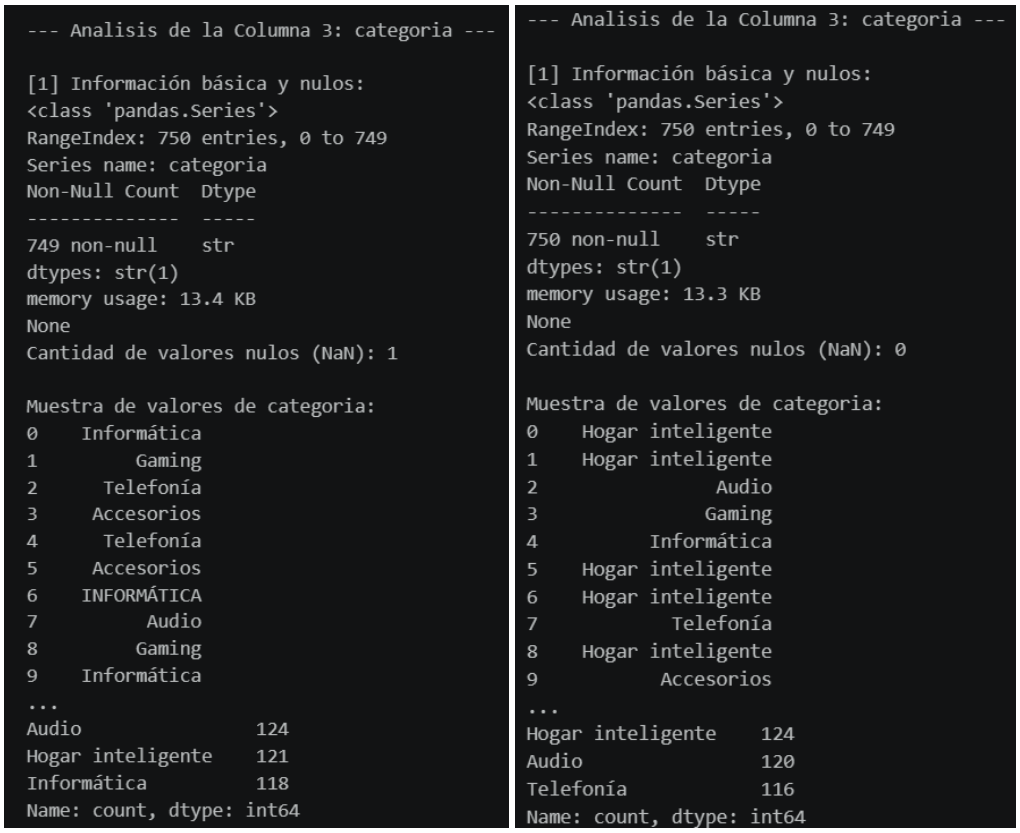


Figura 3: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 4 – Proveedor:

Propósito: Registrar el fabricante o distribuidor oficial asociado a cada producto para la gestión logística de compras, pedidos de reposición y garantías.

Problemas detectados: Se detectan fallos de integridad por valores faltantes en múltiples sedes: Madrid registra 4 valores nulos (NaN) y Sevilla registra 1 valor nulo (NaN). Las sedes de Barcelona y Valencia se encuentran al 100% de su capacidad (750 registros). Adicionalmente, se requiere limpiar posibles espacios en blanco en los nombres comerciales de los proveedores.

Solución: Se aplicará la eliminación de espacios en blanco (.str.strip()) en la totalidad de los archivos y se imputará un valor por defecto (ej. "No especificado") o se gestionará la pérdida del dato en Madrid y Sevilla.

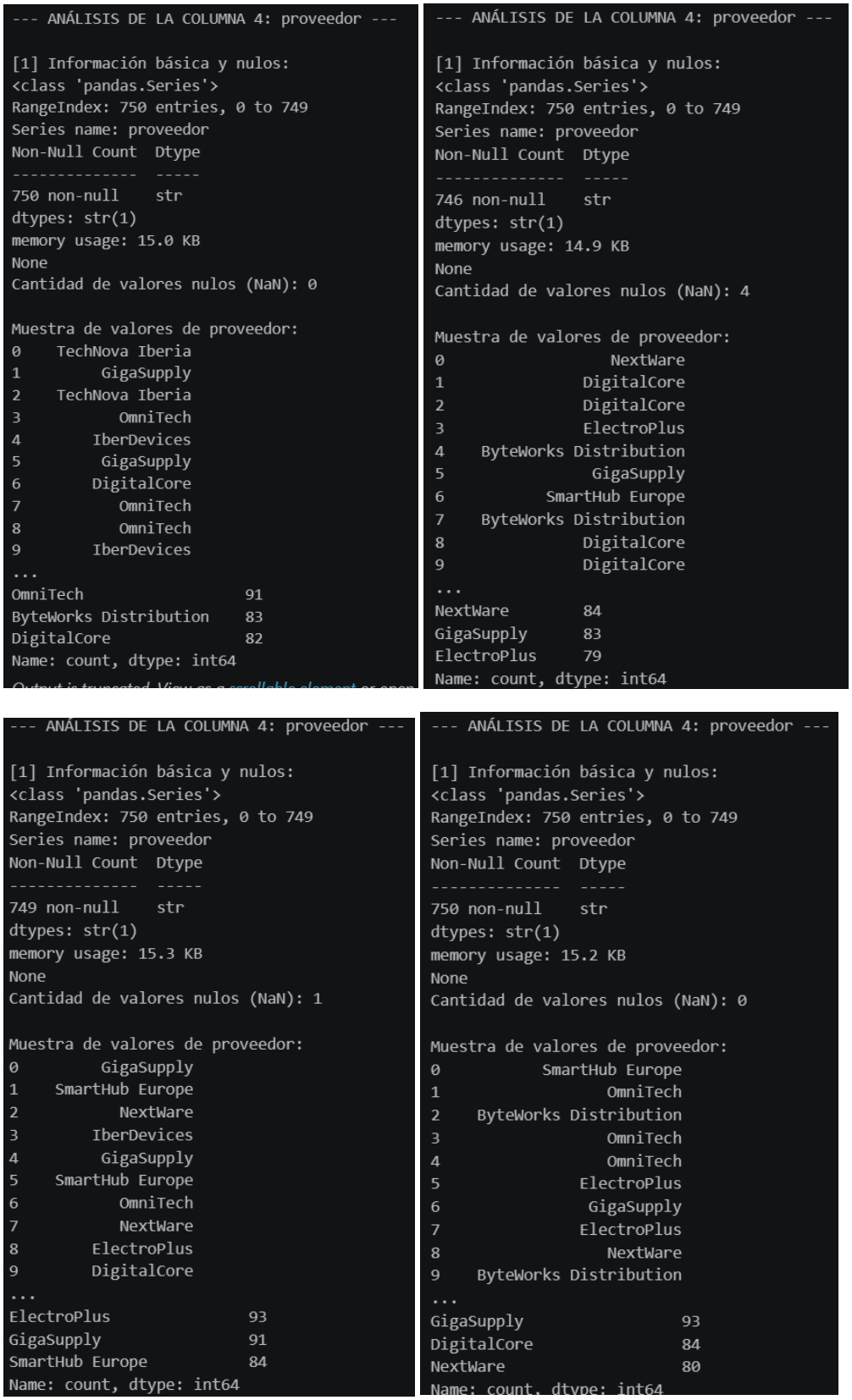


Figura 4: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 5 – Precio Unitario:

Propósito: Representar el valor monetario unitario de cada producto en formato numérico decimal (float64) para la valoración contable y financiera del inventario.

Problemas detectados:

Es una de las columnas con incidencias más críticas del dataset:

- Madrid: Registra 5 valores nulos (NaN) y una anomalía grave de negocio: un precio negativo (-230.55).
- Sevilla: El tipo de dato se ha corrompido a object (cadena de texto) debido a que contiene 1 valor nulo (NaN) y 5 cadenas vacías/espacios en blanco mezcladas con los números.
- Barcelona y Valencia: Presentan valores numéricos válidos (float64) y 0 nulos.

Solución:

Se diseñará una conversión forzada a tipo numérico (pd.to_numeric(errors='coerce')), corrección o conversión a positivo/módulo para valores negativos, e imputación (mediante la media o mediana de la categoría) para los registros nulos y vacíos.

```
--- ANÁLISIS DE LA COLUMNA 5: precio_unitario ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: precio_unitario
Non-Null Count  Dtype
--------------  -----
750 non-null    float64
dtypes: float64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de precio_unitario:
0     767.14
1    1111.89
2     750.46
3     278.63
4     767.67
5    1170.72
6      62.60
7     794.02
8    1359.28
9     933.52
...
50%       763.270000
75%      1138.420000
max      1498.020000
Name: precio_unitario, dtype: float64
```

```
--- ANÁLISIS DE LA COLUMNA 5: precio_unitario ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: precio_unitario
Non-Null Count  Dtype
--------------  -----
745 non-null    str
dtypes: str(1)
memory usage: 10.6 KB
None
Cantidad de valores nulos (NaN): 5

Muestra de valores de precio_unitario:
0     224.42
1    1311.27
2    -230.55
3     295.26
4        NaN
5    1164.87
6    1110.33
7     137.79
8     964.13
9      688.5
...
224.42     1
1311.27    1
-230.55    1
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 5: precio_unitario ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: precio_unitario
Non-Null Count  Dtype
--------------  -----
749 non-null    object
dtypes: object(1)
memory usage: 6.0+ KB
None
Cantidad de valores nulos (NaN): 1
Cantidad de valores vacíos/espacios en blanco: 5

Muestra de valores de precio_unitario:
0    1272.36
1      769.9
2     882.17
3     234.34
4     1240.4
5    1022.15
6    1191.15
7     999.06
8     421.84
...
313.01     2
1272.36    1
769.9      1
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 5: precio_unitario ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: precio_unitario
Non-Null Count  Dtype
--------------  -----
750 non-null    float64
dtypes: float64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de precio_unitario:
0    1196.17
1     579.61
2     469.73
3    1265.17
4     595.91
5    1381.71
6     553.22
7    1402.04
8    1181.19
9      17.05
...
50%       744.205000
75%      1151.020000
max      1497.700000
Name: precio_unitario, dtype: float64
```

Figura 5 : Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 6 – Stock Actual:

Propósito: Cuantificar las unidades físicamente disponibles en las estanterías del almacén para el control de volumen y despacho de pedidos.

Problemas encontrados: Las sedes de Barcelona, Madrid y Valencia presentan una estructura numérica correcta (int64) con 750 registros completos. Sin embargo, en Sevilla, el tipo de dato aparece como object debido a desajustes en el formato de importación de la hoja de cálculo.

Solución: Se estandarizará el tipo de dato a entero numérico (int64) mediante casteo y rellenado de seguridad en los cuatro datasets del diccionario de manera simultánea.

```
--- ANÁLISIS DE LA COLUMNA 6: stock_actual ---

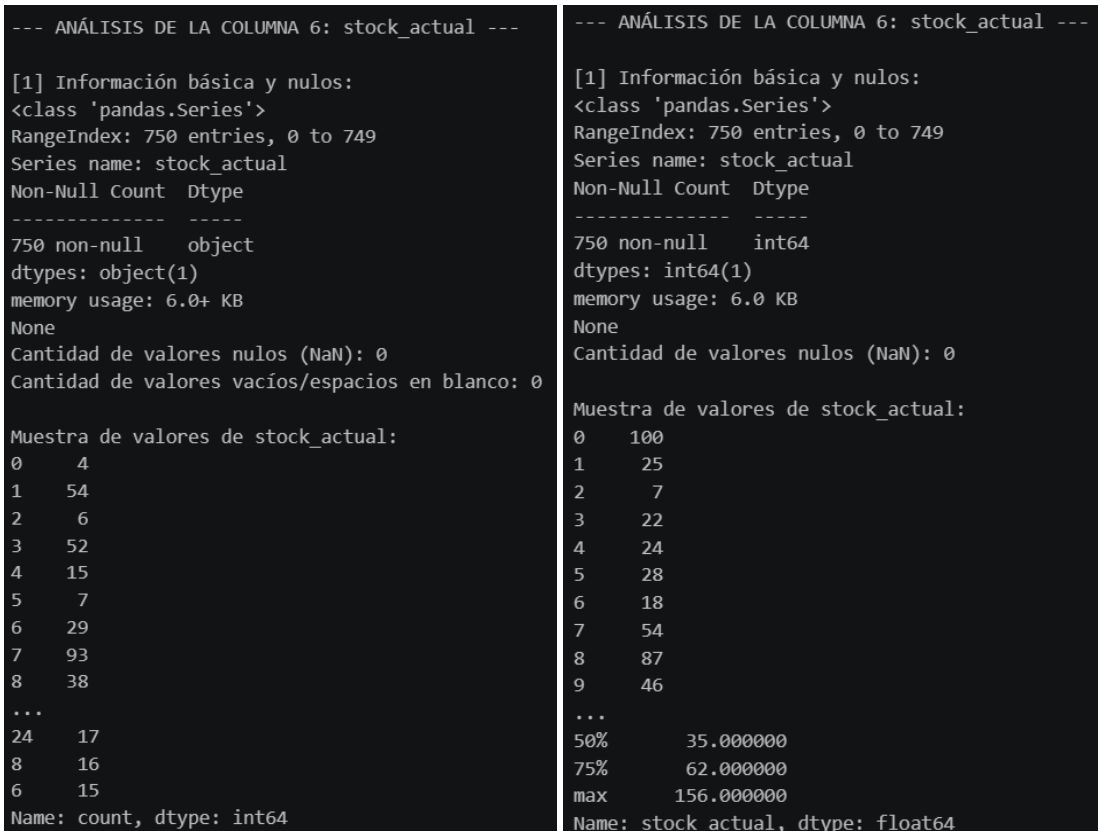
[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_actual
Non-Null Count  Dtype
--------------  -----
750 non-null    int64
dtypes: int64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_actual:
0     83
1    107
2     58
3     74
4    118
5     26
6     17
7     74
8     91
9     40
...
50%       39.000000
75%       64.000000
max      193.000000
Name: stock_actual, dtype: float64
```

```
--- ANÁLISIS DE LA COLUMNA 6: stock_actual ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_actual
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 7.4 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_actual:
0     70
1     16
2     42
3    120
4      1
5     96
6     38
7     65
8     40
9     29
...
20    16
28    14
8     14
Name: count, dtype: int64
```



```
--- ANÁLISIS DE LA COLUMNA 6: stock_actual ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_actual
Non-Null Count  Dtype
--------------  -----
750 non-null    object
dtypes: object(1)
memory usage: 6.0+ KB
None
Cantidad de valores nulos (NaN): 0
Cantidad de valores vacíos/espacios en blanco: 0

Muestra de valores de stock_actual:
0     4
1    54
2     6
3    52
4    15
5     7
6    29
7    93
8    38
...
24    17
8     16
6     15
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 6: stock_actual ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_actual
Non-Null Count  Dtype
--------------  -----
750 non-null    int64
dtypes: int64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_actual:
0    100
1     25
2      7
3     22
4     24
5     28
6     18
7     54
8     87
9     46
...
50%       35.000000
75%       62.000000
max      156.000000
Name: stock_actual, dtype: float64
```

Figura 6 : Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 7 – Stock Mínimo:

Propósito: Establecer la cantidad mínima de seguridad por producto para la activación de alertas automáticas de reabastecimiento y prevención de roturas de stock.

Problemas detectados: En Sevilla se localiza 1 valor nulo (NaN) y la columna adquiere el tipo float64. Madrid, Barcelona y Valencia mantienen la columna completa sin valores nulos.

Solución: La función automatizada rellenará el valor nulo de Sevilla con la mediana del stock mínimo correspondiente a su categoría y casteará la columna a entero (int64).

```
--- ANÁLISIS DE LA COLUMNA 7: stock_minimo ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_minimo
Non-Null Count  Dtype
--------------  -----
750 non-null    int64
dtypes: int64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_minimo:
0    38
1    32
2    16
3    20
4    34
5    13
6    21
7    34
8    31
9    35
...
50%       21.000000
75%       31.000000
max       40.000000
Name: stock_minimo, dtype: float64
```

```
--- ANÁLISIS DE LA COLUMNA 7: stock_minimo ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_minimo
Non-Null Count  Dtype
--------------  -----
750 non-null    int64
dtypes: int64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_minimo:
0    30
1    27
2    18
3    36
4     3
5    27
6    14
7    36
8    14
9    34
...
50%       21.000000
75%       30.000000
max       40.000000
Name: stock_minimo, dtype: float64
```

```
--- ANÁLISIS DE LA COLUMNA 7: stock_minimo ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_minimo
Non-Null Count  Dtype
--------------  -----
749 non-null    float64
dtypes: float64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 1

Muestra de valores de stock_minimo:
0     4.0
1    20.0
2     8.0
3    37.0
4     8.0
5     7.0
6    38.0
7    28.0
8    12.0
9    14.0
...
50%       20.000000
75%       30.000000
max       40.000000
Name: stock_minimo, dtype: float64
```

```
--- ANÁLISIS DE LA COLUMNA 7: stock_minimo ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: stock_minimo
Non-Null Count  Dtype
--------------  -----
750 non-null    int64
dtypes: int64(1)
memory usage: 6.0 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de stock_minimo:
0    34
1    19
2     4
3    11
4     9
5     9
6    15
7    16
8    39
9    38
...
50%       21.000000
75%       31.000000
max       40.000000
Name: stock_minimo, dtype: float64
```

Figura 7: Muestra del análisis de los 4 datasets(BCN-MAD-SEV-VAL)

Columna 8 – Almacen:

Propósito: Identificar de manera unívoca el centro logístico o sede de origen de cada registro (Barcelona, Madrid, Sevilla, Valencia) para garantizar la trazabilidad multisede tras la consolidación.

Problemas encontrados: Todas las sedes presentan 750 registros completos (100% de integridad) con el nombre exacto de su respectiva ciudad. No obstante, al tratarse de un campo clave de segmentación, es susceptible a inconsistencias de espacios sobrantes al fusionar los datos.

Solución: Se aplicará estandarización de texto (.str.strip().str.capitalize()) para asegurar una nomenclatura homogénea en la tabla unificada.

```
--- ANÁLISIS DE LA COLUMNA 8: almacen ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: almacen
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 12.6 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de almacen:
0    Barcelona
1    Barcelona
2    Barcelona
3    Barcelona
4    Barcelona
5    Barcelona
6    Barcelona
7    Barcelona
8    Barcelona
9    Barcelona
...
Resumen de contenido:
almacen
Barcelona    750
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 8: almacen ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: almacen
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 10.4 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de almacen:
0    Madrid
1    Madrid
2    Madrid
3    Madrid
4    Madrid
5    Madrid
6    Madrid
7    Madrid
8    Madrid
9    Madrid
...
Resumen de contenido:
almacen
Madrid    750
Name: count, dtype: int64
```

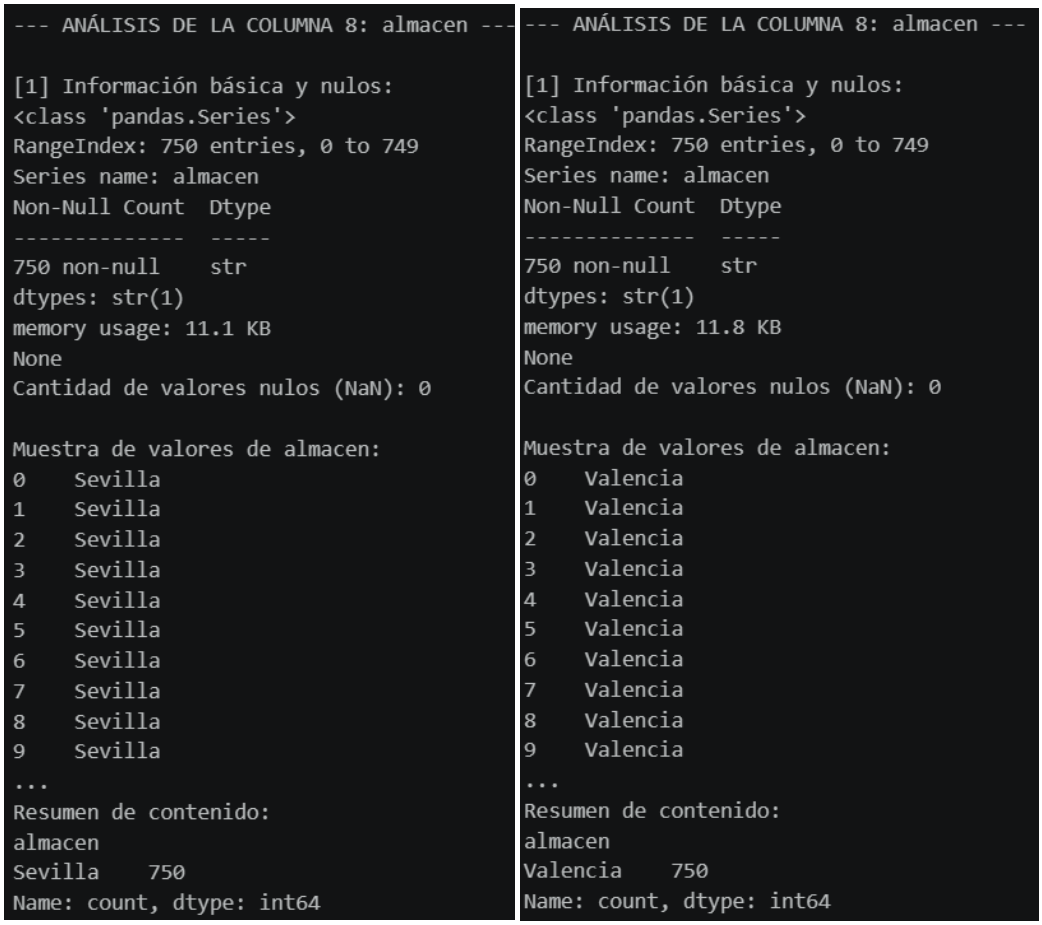


Figura 8: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 9 – Fecha Ultima Entrada:

Propósito: Almacenar la fecha del último movimiento de entrada de mercancía en formato temporal (YYYY-MM-DD) para análisis de rotación, antigüedad de stock y auditoría de inventario.

Problemas detectados: Tanto Madrid como Sevilla contienen 1 valor nulo (NaN) en esta fecha. Barcelona y Valencia están completas. En las cuatro sedes, la columna se importa inicialmente como texto (str/object) en lugar de tipo fecha nativo.

Solución: Se aplicará la conversión automatizada a formato datetime (pd.to_datetime), gestionando los nulos e incoherencias de formato de forma homogénea.

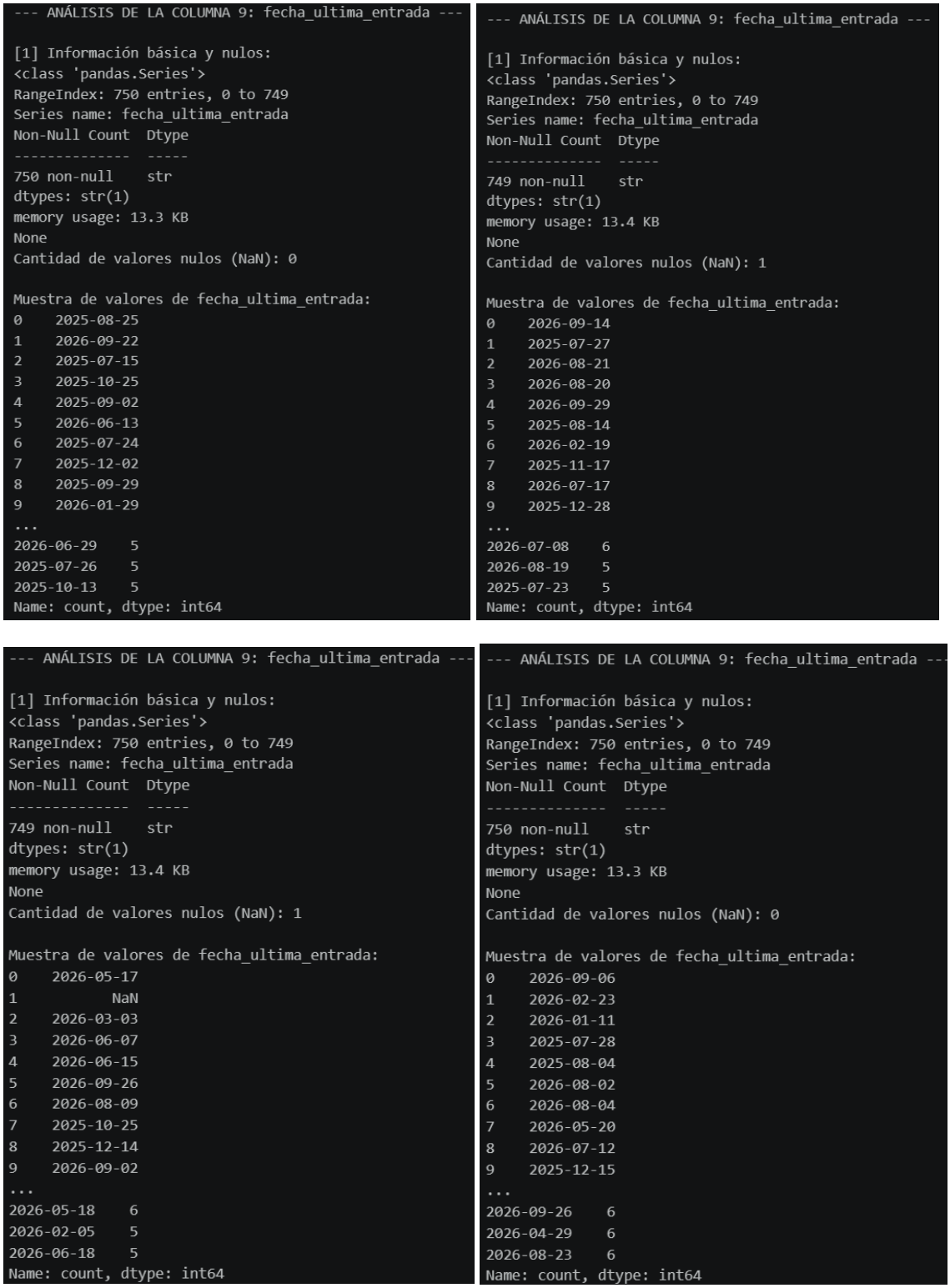


Figura 9: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

Columna 10 – Estado:

Propósito: Indicar el estado operativo del producto (activo, descatalogado, etc.) para determinar su disponibilidad comercial.

Problemas detectados:
Es la columna con mayor nivel de ruido categórico registrado en la auditoría:

- Madrid: Presenta valores con mayúsculas (ACTIVO), datos inconsistentes como X y estados ambiguos como pendiente.
- Sevilla: Registra inconsistencias de capitalización (ACTIVO), valores dudosos como retirado y caracteres no válidos (X).
- Valencia: Contiene 6 registros en mayúsculas (ACTIVO).
- Barcelona: Presenta mezcla de minusculas y mayúsculas (activo, ACTIVO, descatalogado, DESCATALOGADO).

Solución: Se diseñará una regla de normalización estandarizada mediante conversión a minúsculas (.str.lower()), limpieza de espacios, y un mapeo/sustitución de valores inválidos (como X, retirado?, pendiente) hacia las categorías oficiales reconocidas (activo o descatalogado).

```
--- ANÁLISIS DE LA COLUMNA 10: estado ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: estado
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 10.7 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de estado:
0           activo
1           activo
2           activo
3           activo
4           activo
5           activo
6    descatalogado
7           activo
8           activo
9           activo
...
descatalogado     40
ACTIVO             5
DESCATALOGADO      1
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 10: estado ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: estado
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 10.6 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de estado:
0    activo
1    activo
2    activo
3    activo
4    activo
5    activo
6    activo
7    activo
8    activo
9    activo
...
ACTIVO             6
X                  2
pendiente          1
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 10: estado ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: estado
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 10.7 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de estado:
0           activo
1           activo
2           activo
3           activo
4           activo
5           activo
6           activo
7           activo
8           activo
9    descatalogado
...
ACTIVO             6
retirado?          2
X                  1
Name: count, dtype: int64
```

```
--- ANÁLISIS DE LA COLUMNA 10: estado ---

[1] Información básica y nulos:
<class 'pandas.Series'>
RangeIndex: 750 entries, 0 to 749
Series name: estado
Non-Null Count  Dtype
--------------  -----
750 non-null    str
dtypes: str(1)
memory usage: 10.6 KB
None
Cantidad de valores nulos (NaN): 0

Muestra de valores de estado:
0    activo
1    activo
2    activo
3    activo
4    activo
5    activo
6    activo
7    activo
8    activo
9    activo
...
activo           708
descatalogado     36
ACTIVO             6
Name: count, dtype: int64
```

Figura 10: Muestra del análisis de los 4 datasets (BCN-MAD-SEV-VAL)

4.- Fase de limpieza Automática Multisede

4.1.- Propósito y Estrategia de Limpieza:

Tras el diagnóstico realizado en la fase de auditoría, se constató que cada uno de los cuatro ficheros regionales (Barcelona, Madrid, Sevilla y Valencia) presentaba anomalías heterogéneas que impedían la consulta directa y la consolidación fiable del inventario.

Para resolver esta problemática de forma escalable y mantenible, se diseñó un pipeline automatizado de limpieza en Python basado en la función limpiar_inventario(). Esta función itera de manera centralizada sobre la estructura de diccionario que contiene los DataFrames, garantizando que todos los registros pasen por las mismas reglas de validación y transformación sin modificar los archivos de origen.

4.2.- Reglas de Transformación por Columna:

La lógica implementada en la función de limpieza responde punto por punto a los hallazgos documentados en el informe:

- product_id (Identificador Único): Se eliminaron los registros con identificador nulo (NaN) y se aplicó la eliminación de duplicados exactos dentro de cada sede (keep='first'), restituyendo la clave primaria del inventario.
- nombre y proveedor (Campos de Texto Libre): Se gestionaron los valores nulos mediante la asignación de literales por defecto ("Producto no especificado" / "Proveedor no asignado") y se aplicó una limpieza estricta de espacios en blanco iniciales y finales (.str.strip()).
- categoria (Estandarización Categórica): Se corrigieron las diferencias de capitalización (como términos completamente en mayúsculas) mediante la normalización a formato título (.str.capitalize()).
- precio_unitario (Tratamiento Financiero y Numérico): Se forzó la conversión a tipo flotante (pd.to_numeric), convirtiendo en nulos las cadenas vacías o con espacios de Sevilla. Los precios negativos identificados en Madrid se corrigieron mediante su valor absoluto (.abs()), y los nulos se imputaron con la mediana representativa redondeada a dos decimales.
- stock_actual y stock_minimo (Unidades Físicas): Se estandarizó el tipo de dato a entero (int64). Los valores negativos o incoherentes se ajustaron al límite inferior lógico (0), imputando los faltantes mediante la mediana.

- fecha_ultima_entrada (Formato Temporal): Se convirtió la columna a tipo de dato fecha nativo (pd.to_datetime), permitiendo operaciones de filtrado temporal.
- estado (Normalización de Estados de Negocio): Se estandarizaron las cadenas a minúsculas (.str.lower()) y se aplicó un mapa de homologación categórica que transformó valores ambiguos o erróneos ('X', 'retirado?', 'pendiente') hacia los estados corporativos oficiales (activo o descatalogado).

```
========================================
  Iniciando limpieza de la sede: BCN
========================================
✅ Sede BCN limpiada correctamente.
   Filas iniciales: 750 | Filas tras eliminar duplicados/nulos en ID: 750
   Nulos restantes totales: 0

========================================
  Iniciando limpieza de la sede: MAD
========================================
✅ Sede MAD limpiada correctamente.
   Filas iniciales: 750 | Filas tras eliminar duplicados/nulos en ID: 742
   Nulos restantes totales: 5

========================================
  Iniciando limpieza de la sede: SEV
========================================
✅ Sede SEV limpiada correctamente.
   Filas iniciales: 750 | Filas tras eliminar duplicados/nulos en ID: 743
   Nulos restantes totales: 5

========================================
  Iniciando limpieza de la sede: VAL
========================================
✅ Sede VAL limpiada correctamente.
   Filas iniciales: 750 | Filas tras eliminar duplicados/nulos en ID: 750
   Nulos restantes totales: 0
```

Figura 11: Muestra de la limpieza de los 4 datasets(BCN, MAD,SEV, VAL)

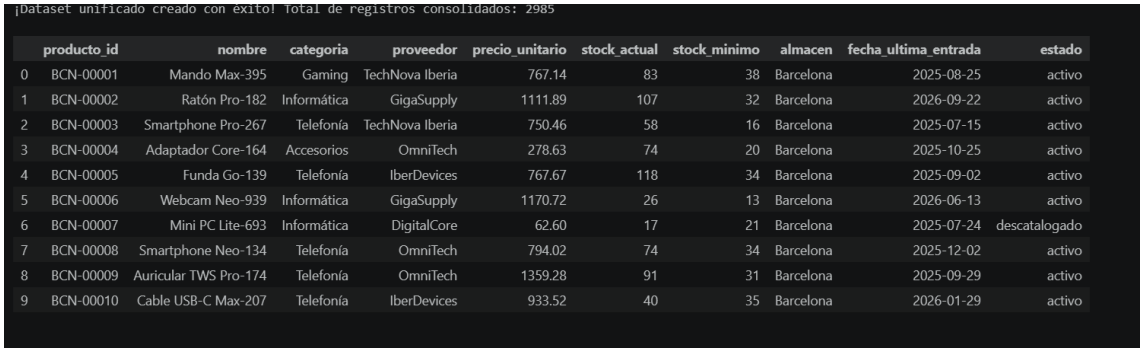
5.- Unificación e Integración del dataset Maestro

5.1.- Concatenación y Trazabilidad Multisede:

Una vez finalizada la limpieza individual de cada dataset regional, se procedió a la integración vertical de la información para dar lugar a una base de datos global unificada.

Para garantizar la trazabilidad multisede, se preservó la columna almacen con la localización de origen (Barcelona, Madrid, Sevilla, Valencia). La consolidación se ejecutó mediante la función pd.concat() reindexando las filas para formar una tabla continua.

5.2.- Resultado del Dataset Consolidado:



¡Dataset unificado creado con éxito! Total de registros consolidados: 2985

	producto_id	nombre	categoria	proveedor	precio_unitario	stock_actual	stock_minimo	almacen	fecha_ultima_entrada	estado
0	BCN-00001	Mando Max-395	Gaming	TechNova Iberia	767.14	83	38	Barcelona	2025-08-25	activo
1	BCN-00002	Ratón Pro-182	Informática	GigaSupply	1111.89	107	32	Barcelona	2026-09-22	activo
2	BCN-00003	Smartphone Pro-267	Telefonía	TechNova Iberia	750.46	58	16	Barcelona	2025-07-15	activo
3	BCN-00004	Adaptador Core-164	Accesorios	OmniTech	278.63	74	20	Barcelona	2025-10-25	activo
4	BCN-00005	Funda Go-139	Telefonía	IberDevices	767.67	118	34	Barcelona	2025-09-02	activo
5	BCN-00006	Webcam Neo-939	Informática	GigaSupply	1170.72	26	13	Barcelona	2026-06-13	activo
6	BCN-00007	Mini PC Lite-693	Informática	DigitalCore	62.60	17	21	Barcelona	2025-07-24	descatalogado
7	BCN-00008	Smartphone Neo-134	Telefonía	OmniTech	794.02	74	34	Barcelona	2025-12-02	activo
8	BCN-00009	Auricular TWS Pro-174	Telefonía	OmniTech	1359.28	91	31	Barcelona	2025-09-29	activo
9	BCN-00010	Cable USB-C Max-207	Telefonía	IberDevices	933.52	40	35	Barcelona	2026-01-29	activo

Figura 12: Muestra del dataset unificado